`async_scope` – Creating scopes for non-sequential concurrency

Document #:	P3149R3
Date:	2024-05-21
Project:	Programming Language C++
Audience:	SG1 Parallelism and Concurrency LEWG Library Evolution

Reply-to:	Ian Petersen <ispeters@meta.com> Ján Ondrušek <ondrusek@meta.com> Jessica Wong <jesswong@meta.com> Kirk Shoop <kirk.shoop@gmail.com> Lee Howes <lwh@fb.com> Lucian Radu Teodorescu <lucteo@lucteo.ro>

Contents

§ 1 Changes

§ 1.1 R3

- Update slide code to be exception safe
- Split the async scope concept into a scope and token; update `counting_scope` to match
- Rename `counting_scope` to `simple_counting_scope` and give the name `counting_scope` to a scope with a stop source
- Add example for recursively spawned work using `let_with_async_scope` and `counting_scope`

§ 1.2 R2

- Update `counting_scope::nest()` to explain when the scope's count of outstanding senders is decremented and remove `counting_scope::joined()`, `counting_scope::join_started()`, and `counting_scope::use_count()` on advice of SG1 straw poll:

forward P3149R1 to LEWG for inclusion in C++26 after P2300 is included in C++26, with notes:

1. the point of refcount decrement to be moved after the child operation state is destroyed
2. a future paper should explore the design for cancellation of scopes
3. observers (joined, join_started, use_count) can be removed

SF	F	N	A	SA
10	14	2	0	1

Consensus

SA: we are moving something without wide implementation experience, the version with experience has cancellation of scopes

— Add a fourth state to `counting_scope` so that it can be used as a data-member safely

§ 1.3 R1

— Add implementation experience

— Incorporate pre-meeting feedback from Eric Niebler

§ 1.4 R0

— First revision

§ 2 Introduction

[P2300R7] lays the groundwork for writing structured concurrent programs in C++ but it leaves three important scenarios under- or unaddressed:

1. progressively structuring an existing, unstructured concurrent program;
2. starting a dynamic number of parallel tasks without "losing track" of them; and
3. opting in to eager execution of sender-shaped work when appropriate.

This paper describes the utilities needed to address the above scenarios within the following constraints:

— *No detached work by default;* as specified in [P2300R7], the `start_detached` and `ensure_started` algorithms invite users to start concurrent work with no built-in way to know when that work has finished.

  — Such so-called "detached work" is undesirable; without a way to know when detached work is done, it is difficult know when it is safe to destroy any resources referred to by the work. Ad hoc solutions to this shutdown problem add unnecessary complexity that can be avoided by ensuring all concurrent work is "attached".

  — [P2300R7]'s introduction of structured concurrency to C++ will make async programming with C++ much easier but experienced C++ programmers typically believe that async C++ is "just hard" and that starting async work *means* starting detached work (even if they are not thinking about the distinction between attached and detached work) so adapting to a post-[P2300R7] world will require unlearning many deprecated patterns. It is thus useful as a teaching aid to remove the unnecessary temptation of falling back on old habits.

— *No dependencies besides [P2300R7];* it will be important for the success of [P2300R7] that existing code bases can migrate from unstructured concurrency to structured concurrency in an incremental way so tools for progressively structuring code should not take on risk in the form of unnecessary dependencies.

The proposed solution comes in the following parts:

— `template <class Token, class Sender> concept async_scope_token;`

— `template <class Scope> concept async_scope;`

— `sender auto nest(sender auto&& snd, async_scope_token auto token);`

— `void spawn(sender auto&& snd, async_scope_token auto token, auto&& env);`

— `sender auto spawn_future(sender auto&& snd, async_scope_token auto token, auto&& env);`

— Proposed in [P3296R0]: `sender auto let_with_async_scope(callable auto&& senderFactory);`

— `struct simple_counting_scope;` and

— `struct counting_scope.`

§ 2.1 Implementation experience

The general concept of an async scope to manage work has been deployed broadly at Meta. Code written with Folly's coroutine library, [`folly::coro`], uses [`folly::coro::AsyncScope`] to safely launch awaitables. Most code written with Unifex, an implementation of an earlier version of the *Sender/Receiver* model proposed in [P2300R7], uses [`unifex::v1::async_scope`], although experience with the v1 design led to the creation of [`unifex::v2::async_scope`], which has a smaller interface and a cleaner definition of responsibility.

As an early adopter of Unifex, [rsys] (Meta's cross-platform voip client library) became the entry point for structured concurrency in mobile code at Meta. We originally built rsys with an unstructured asynchrony model built around posting callbacks to threads in order to optimize for binary size. However, this came at the expense of developer velocity due to the increasing cost of debugging deadlocks and crashes resulting from race conditions.

We decided to adopt Unifex and refactor towards a more structured architecture to address these problems systematically. Converting an unstructured production codebase to a structured one is such a large project that it needs to be done in phases. As we began to convert callbacks to senders/tasks, we quickly realized that we needed a safe place to start structured asynchronous work in an unstructured environment. We addressed this need with `unifex::v1::async_scope` paired with an executor to address a recurring pattern:

Before

```
// Abstraction for thread that has the ability
// to execute units of work.
class Executor {
 public:
  virtual void add(Func function) noexcept = 0;
};

// Example class
class Foo {
  std::shared_ptr<Executor> exec_;

 public:
  void doSomething() {
     auto asyncWork = [&]() {
       // do something
     };
     exec_->add(asyncWork);
  }
};
```

After

```
// Utility class for executing async work on an
// async_scope and on the provided executor
class ExecutorAsyncScopePair {
  unifex::v1::async_scope scope_;
  ExecutorScheduler exec_

 public:
  void add(Func func) {
    scope_.detached_spawn_call_on(
      exec_, func);
  }

  auto cleanup() {
    return scope_.cleanup();
  }
};

// Example class
class Foo {
 std::shared_ptr<ExecutorAsyncScopePair> exec_;

 public:
  ~Foo() {
    sync_wait(exec_->cleanup());
  }

  void doSomething() {
    auto asyncWork = [&]() {
      // do something
    };


   exec_->add(asyncWork);
  }
};
```

This broadly worked but we discovered that the above design coupled with the v1 API allowed for too many redundancies and conflated too many responsibilities (scoping async work, associating work with a stop source, and transferring scoped work to a new scheduler).

We learned that making each component own a distinct responsibility will minimize the confusion and increase the structured concurrency adoption rate. The above example was an intuitive use of async_scope because the concept of a "scoped executor" was familiar to many engineers and is a popular async pattern in other programming languages. However, the above design abstracted away some of the APIs in async_scope that explicitly asked for a scheduler, which would have helped challenge the assumption engineers made about async_scope being an instance of a "scoped executor".

Cancellation was an unfamiliar topic for engineers within the context of asynchronous programming. The `v1::async_scope` provided both `cleanup()` and `complete()` to give engineers the freedom to decide between canceling work or waiting for work to finish. The different nuances on when this should happen and how it happens ended up being an obstacle that engineers didn't want to deal with.

Over time, we also found redundancies in the way `v1::async_scope` and other algorithms were implemented and identified other use cases that could benefit from a different kind of async scope. This motivated us to create `v2::async_scope` which only has one responsibility (scope), and `nest` which helped us improve maintainability and flexibility of Unifex.

The unstructured nature of `cleanup()`/`complete()` in a partially structured codebase introduced deadlocks when engineers nested the `cleanup()`/`complete()` sender in the scope being joined. This risk of deadlock remains with `v2::async_scope::join()` however, we do think this risk can be managed and is worth the tradeoff in exchange for a more coherent architecture that has fewer crashes. For example, we have experienced a significant reduction in these types of deadlocks once engineers understood that `join()` is a destructor-like operation that needs to be run only by the scope's owner. Since there is no language support to manage async lifetimes automatically, this insight was key in preventing these types of deadlocks. Although this breakthrough was a result of strong guidance from experts, we believe that the simpler design of `v2::async_scope` would make this a little easier.

We strongly believe that async_scope was necessary for making structured concurrency possible within rsys, and we believe that the improvements we made with `v2::async_scope` will make the adoption of P2300 more accessible.

§ 3 Motivation

§ 3.1 Motivating example

Let us assume the following code:

```
namespace ex = std::execution;

struct work_context;
struct work_item;
void do_work(work_context&, work_item*);
std::vector<work_item*> get_work_items();

int main() {
    static_thread_pool my_pool{8};
    work_context ctx; // create a global context for the application

    std::vector<work_item*> items = get_work_items();
    for (auto item : items) {
        // Spawn some work dynamically
        ex::sender auto snd = ex::transfer_just(my_pool.get_scheduler(), item) |
                              ex::then([&](work_item* item) { do_work(ctx, item); });
        ex::start_detached(std::move(snd));
    }
    // `ctx` and `my_pool` are destroyed
}
```

In this example we are creating parallel work based on the given input vector. All the work will be spawned on the local `static_thread_pool` object, and will use a shared `work_context` object.

Because the number of work items is dynamic, one is forced to use `start_detached()` from [P2300R7] (or something equivalent) to dynamically spawn work. [P2300R7] doesn't provide any facilities to spawn dynamic work and return a sender (i.e., something like `when_all` but with a dynamic number of input senders).

Using `start_detached()` here follows the *fire-and-forget* style, meaning that we have no control over, or awareness of, the completion of the async work that is being spawned.

At the end of the function, we are destroying the `work_context` and the `static_thread_pool`. But at that point, we don't know whether all the spawned async work has completed. If any of the async work is incomplete, this might lead to crashes.

[P2300R7] doesn't give us out-of-the-box facilities to use in solving these types of problems.

This paper proposes the `counting_scope` and [P3296R0]'s `let_with_async_scope` facilities that would help us avoid the invalid behavior. With `counting_scope`, one might write safe code this way:

```
namespace ex = std::execution;

struct work_context;
struct work_item;
void do_work(work_context&, work_item*);
std::vector<work_item*> get_work_items();

int main() {
    static_thread_pool my_pool{8};
    work_context ctx;         // create a global context for the application
    ex::counting_scope scope; // create this *after* the resources it protects

    // make sure we always join
    unifex:scope_guard join = [&]() noexcept {
      // wait for all nested work to finish
      this_thread::sync_wait(scope.join()); // NEW!
    };
```

```
    std::vector<work_item*> items = get_work_items();
    for (auto item : items) {
      // Spawn some work dynamically
      ex::sender auto snd = ex::transfer_just(my_pool.get_scheduler(), item) |
                            ex::then([&](work_item* item) { do_work(ctx, item); });

      // start `snd` as before, but associate the spawned work with `scope` so that it can
      // be awaited before destroying the resources referenced by the work (i.e. `my_pool`
      // and `ctx`)
      ex::spawn(std::move(snd), scope.get_token()); // NEW!
    }

    // `ctx` and `my_pool` are destroyed *after* they are no longer referenced
}
```

With [P3296R0]'s `let_with_async_scope`, one might write safe code this way:

```
namespace ex = std::execution;

struct work_context;
struct work_item;
void do_work(work_context&, work_item*);
std::vector<work_item*> get_work_items();

int main() {
  static_thread_pool my_pool{8};
  work_context ctx;         // create a global context for the application

  this_thread::sync_wait(ex::let_with_async_scope(ex::just(get_work_items()), [&](auto scope) {
    for (auto item : items) {
      // Spawn some work dynamically
      ex::sender auto snd = ex::transfer_just(my_pool.get_scheduler(), item) |
                            ex::then([&](work_item* item) { do_work(ctx, item); });

      // start `snd` as before, but associate the spawned work with `scope` so that it can
      // be awaited before destroying the resources referenced by the work (i.e. `my_pool`
      // and `ctx`)
      ex::spawn(std::move(snd), scope); // NEW!
    }
    return just();
  }));

  // `ctx` and `my_pool` are destroyed *after* they are no longer referenced
}
```

Simplifying the above into something that fits in a Tony Table to highlight the differences gives us:

Before

```
namespace ex = std::execution;

struct context;
ex::sender auto work(const context&);

int main() {
  context ctx;

  ex::sender auto snd = work(ctx);

  // fire and forget
  ex::start_detached(std::move(snd));

  // `ctx` is destroyed, perhaps
        before
  // `snd` is done
}
```

With `counting_scope`

```
namespace ex = std::execution;

struct context;
ex::sender auto work(const context&);

int main() {
  context ctx;
  ex::counting_scope scope;

  ex::sender auto snd = work(ctx);

  try {
      // fire, but don't forget
      ex::spawn(std::move(snd),
        scope.get_token());
  } catch (...) {
      // do something to handle
        exception
  }

  // wait for all work nested within
        scope
  // to finish
  this_thread::sync_wait(scope.join());

  // `ctx` is destroyed once nothing
  // references it
}
```

With `let_with_async_scope`

```
namespace ex = std::execution;

struct context;
ex::sender auto work(const
        context&);

int main() {
  context ctx;
  this_thread::sync_wait(ex::just()
      | ex::let_with_async_scope([&]
        (auto scope) {
        ex::sender auto snd =
        work(ctx);

        // fire, but don't forget
        ex::spawn(std::move(snd),
        scope.get_token());
      }));

  // `ctx` is destroyed once nothing
  // references it
}
```

Please see below for more examples.

§ 3.2 `counting_scope` and `let_with_async_scope` are a step forward towards Structured Concurrency

Structured Programming [Dahl72] transformed the software world by making it easier to reason about the code, and build large software from simpler constructs. We want to achieve the same effect on concurrent programming by ensuring that we *structure* our concurrent code. [P2300R7] makes a big step in that direction, but, by itself, it doesn't fully realize the principles of Structured Programming. More specifically, it doesn't always ensure that we can apply the *single entry, single exit point* principle.

The `start_detached` sender algorithm fails this principle by behaving like a `GOTO` instruction. By calling `start_detached` we essentially continue in two places: in the same function, and on different thread that executes the given work. Moreover, the lifetime of the work started by `start_detached` cannot be bound to the local context. This will prevent local reasoning, which will make the program harder to understand.

To properly structure our concurrency, we need an abstraction that ensures that all async work that is spawned has a defined, observable, and controllable lifetime. This is the goal of `counting_scope` and `let_with_async_scope`.

§ 4 Examples of use

§ 4.1 Spawning work from within a task

Use `let_with_async_scope` in combination with a `system_context` from [P2079R2] to spawn work from within a task:

```
namespace ex = std::execution;

int main() {
    ex::system_context ctx;
    int result = 0;

    ex::scheduler auto sch = ctx.scheduler();

    ex::sender auto val = ex::just()
        | ex::let_with_async_scope([sch](ex::async_scope_token auto scope) {
          int val = 13;

          auto print_sender = ex::just()
              | ex::then([val] {
                std::cout << "Hello world! Have an int with value: " << val << "\n";
              });

          // spawn the print sender on sch
          //
          // NOTE: if spawn throws, let_with_async_scope will capture the exception
          //       and propagate it through its set_error completion
          ex::spawn(ex::on(sch, std::move(print_sender)), scope);

          return ex::just(val);
        }))
        | ex::then([&result](auto val) { result = val });

    this_thread::sync_wait(ex::on(sch, std::move(val)));

    std::cout << "Result: " << result << "\n";
}

// 'let_with_async_scope' ensures that, if all work is completed successfully, the result will be 13
// `sync_wait` will throw whatever exception is thrown by the callable passed to `let_with_async_scope`
```

§ 4.2 Starting work nested within a framework

In this example we use the `counting_scope` within a class to start work when the object receives a message and to wait for that work to complete before closing.

```
namespace ex = std::execution;

struct my_window {
    class close_message {};

    ex::sender auto some_work(int message);

    ex::sender auto some_work(close_message message);
```

```
    void onMessage(int i) {
        ++count;
        ex::spawn(ex::on(sch, some_work(i)), scope);
    }

    void onClickClose() {
        ++count;
        ex::spawn(ex::on(sch, some_work(close_message{})), scope);
    }

    my_window(ex::system_scheduler sch, ex::counting_scope::token scope)
      : sch(sch), scope(scope) {
      // register this window with the windowing framework somehow so that
      // it starts receiving calls to onClickClose() and onMessage()
    }

    ex::system_scheduler sch;
    ex::counting_scope::token scope;
    int count{0};
};

int main() {
    // keep track of all spawned work
    ex::counting_scope scope;
    ex::system_context ctx;
    try {
        my_window window{ctx.get_scheduler(), scope.get_token()};
    } catch (...) {
      // do something with exception
    }
    // wait for all work nested within scope to finish
    this_thread::sync_wait(scope.join());
    // all resources are now safe to destroy
    return window.count;
}
```

§ 4.3 Starting parallel work

In this example we use `let_with_async_scope` to construct an algorithm that performs parallel work. Here `foo` launches 100 tasks that concurrently run on some scheduler provided to `foo`, through its connected receiver, and then the tasks are asynchronously joined. This structure emulates how we might build a parallel algorithm where each `some_work` might be operating on a fragment of data.

```
namespace ex = std::execution;

ex::sender auto some_work(int work_index);

ex::sender auto foo(ex::scheduler auto sch) {
  return ex::just()
      | ex::let_with_async_scope([sch](ex::async_scope_token auto scope) {
        return ex::schedule(sch)
            | ex::then([] {
              std::cout << "Before tasks launch\n";
            })
            | ex::then([=] {
              // Create parallel work
              for (int i = 0; i < 100; ++i) {
                // NOTE: if spawn() throws, the exception will be propagated as the
                //       result of let_with_async_scope through its set_error completion
                ex::spawn(ex::on(sch, some_work(i)), scope);
              }
            });
      })
      | ex::then([] { std::cout << "After tasks complete successfully\n"; });
}
```

§ 4.4 Listener loop in an HTTP server

This example shows how one can write the listener loop in an HTTP server, with the help of coroutines. The HTTP server will continuously accept new connection and start work to handle the requests coming on the new connections. While the listening activity is bound in the scope of the loop, the lifetime of handling requests may exceed the scope of the loop. We use `counting_scope` to limit the lifetime of the request handling without blocking the acceptance of new requests.

```
namespace ex = std::execution;

task<size_t> listener(int port, io_context& ctx, static_thread_pool& pool) {
    size_t count{0};
    listening_socket listen_sock{port};
```

```
    co_await ex::let_with_async_scope(
        ex::just(), [&](ex::async_scope_token auto scope) -> task<void> {
          while (!ctx.is_stopped()) {
            // Accept a new connection
            connection conn = co_await async_accept(ctx, listen_sock);
            count++;

            // Create work to handle the connection in the scope of `work_scope`
            conn_data data{std::move(conn), ctx, pool};
            ex::sender auto snd = ex::just(std::move(data))
                | ex::let_value([](auto& data) {
                  return handle_connection(data);
                });

            ex::spawn(std::move(snd), scope);
          }
        });

    // At this point, all the request handling is complete
    co_return count;
}
```

[libunifex] has a very similar example HTTP server at [io_uring HTTP server] that compiles and runs on Linux-based machines with `io_uring` support.

§ 4.5 Pluggable functionality through composition

This example is based on real code in rsys, but it reduces the real code to slideware and ports it from Unifex to the proposed `std::execution` equivalents. The central abstraction in rsys is a `Call`, but each integration of rsys has different needs so the set of features supported by a `Call` varies with the build configuration. We support this configurability by exposing the equivalent of the following method on the `Call` class:

```
template <typename Feature>
Handle<Feature> Call::get();
```

and it's used like this in app-layer code:

```
unifex::task<void> maybeToggleCamera(Call& call) {
  Handle<Camera> camera = call.get<Camera>();

  if (camera) {
    co_await camera->toggle();
  }
}
```

A `Handle<Feature>` is effectively a part-owner of the `Call` it came from.

The team that maintains rsys and the teams that use rsys are, unsurprisingly, different teams so rsys has to be designed to solve organizational problems as well as technical problems. One relevant design decision the rsys team made is that it is safe to keep using a `Handle<Feature>` after the end of its `Call`'s lifetime; this choice adds some complexity to the design of `Call` and its various features but it also simplifies the support relationship between the rsys team and its many partner teams because it eliminates many crash-at-shutdown bugs.

```
namespace rsys {

class Call {
 public:
  unifex::nothrow_task<void> destroy() noexcept {
    // first, close the scope to new work and wait for existing work to finish
    scope_->close();
    co_await scope_->join();

    // other clean-up tasks here
  }

  template <typename Feature>
  Handle<Feature> get() noexcept;

 private:
  // an async scope shared between a call and its features
  std::shared_ptr<std::execution::counting_scope> scope_;
  // each call has its own set of threads
  ExecutionContext context_;

  // the set of features this call supports
```

```
  FeatureBag features_;
};

class Camera {
 public:
  std::execution::sender auto toggle() {
    namespace ex = std::execution;

    return ex::just() | ex::let_value([this]() {
      // this callable is only invoked if the Call's scope is in
      // the open or unused state when nest() is invoked, making
      // it safe to assume here that:
      //
      //  - scheduler_ is not a dangling reference to the call's
      //    execution context
      //  - Call::destroy() has not progressed past starting the
      //    join-sender so all the resources owned by the call
      //    are still valid
      //
      // if the nest() attempt fails because the join-sender has
      // started (or even if the Call has been completely destroyed)
      // then the sender returned from toggle() will safely do
      // nothing before completing with set_stopped()

      return ex::schedule(scheduler_) | ex::then([this]() {
        // toggle the camera
      });
    }) | ex::nest(callScope_->get_token());
  }

 private:
  // a copy of this camera's Call's scope_ member
  std::shared_ptr<ex::counting_scope> callScope_;
  // a scheduler that refers to this camera's Call's ExecutionContext
  Scheduler scheduler_;
};

}
```

§ 4.6 Recursively spawning work until completion

Below are three ways you could recursively spawn work on a scope using `let_with_async_scope` or `counting_scope`.

§ 4.6.1 `let_with_async_scope` with `spawn()`

```
struct tree {
  std::unique_ptr<tree> left;
  std::unique_ptr<tree> right;
  int data;
};

auto process(ex::scheduler auto sch, auto scope, tree& t) noexcept {
  return ex::schedule(sch) | then([sch, &]() {
    if (t.left)
      ex::spawn(process(sch, scope, t.left.get()), scope);
    if (t.right)
      ex::spawn(process(sch, scope, t.right.get()), scope);
    do_stuff(t.data);
  }) | ex::let_error([](auto& e) {
    // log error
    return just();
  });
}

int main() {
  ex::scheduler sch;
  tree t = make_tree();
  // let_with_async_scope will ensure all new work will be spawned on the
  // scope and will not be joined until all work is finished.
  // NOTE: Exceptions will not be surfaced to let_with_async_scope; exceptions
  // will be handled by let_error instead.
  this_thread::sync_wait(ex::let_with_async_scope([&, sch](auto scope) {
    return process(sch, scope, t);
  }));
}
```

§ 4.6.2 `let_with_async_scope` with `spawn_future()`

```
struct tree {
  std::unique_ptr<tree> left;
  std::unique_ptr<tree> right;
  int data;
};

auto process(ex::scheduler auto sch, auto scope, tree& t) {
    return ex::schedule(sch) | ex::let_value([sch, &]() {
      unifex::any_sender_of<> leftFut = ex::just();
      unifex::any_sender_of<> rightFut = ex::just();
      if (t.left) {
         leftFut = ex::spawn_future(
         scope, process(sch, scope, t.left.get()));
      }

      if (t.right) {
         rightFut = ex::spawn_future(
         scope, process(sch, scope, t.right.get()));
      }

      do_stuff(t.data);
      return ex::when_all(leftFut, rightFut) | ex::then([](auto&&...) noexcept {});
    });
}

int main() {
    ex::scheduler sch;
    tree t = make_tree();
    // let_with_async_scope will ensure all new work will be spawned on the
    // scope and will not be joined until all work is finished
    // NOTE: Exceptions will be surfaced to let_with_async_scope which will
    // call set_error with the exception_ptr
    this_thread::sync_wait(ex::let_with_async_scope([&, sch](auto scope) {
        return process(sch, scope, t);
    }));
}
```

4.6.3 `counting_scope`

```
struct tree {
  std::unique_ptr<tree> left;
  std::unique_ptr<tree> right;
  int data;
};

auto process(ex::counting_scope_token scope, ex::scheduler auto sch, tree& t) noexcept {
  return ex::schedule(sch) | ex::then([sch, &]() noexcept {
    if (t.left)
       ex::spawn(process(sch, t.left.get()), scope);

    if (t.right)
       ex::spawn(process(sch, t.right.get()), scope);

     do_stuff(t.data);
   }) | ex::let_error([](auto& e) {
       // log error
       return just();
   });
}

int main() {
  ex::scheduler sch;
  tree t = make_tree();
  ex::counting_scope scope;
  this_thread::sync_wait(process(scope.get_token(), sch, t));
  this_thread::sync_wait(scope.join());
}
```

5 Async Scope, usage guide

An async scope is a type that implements a “bookkeeping policy” for senders that have been `nest()`ed within the scope. Depending on the policy, different guarantees can be provided in terms of the lifetimes of the scope and any nested senders. The `counting_scope` described in this paper defines a policy that has proven useful while progressively adding structure to existing, unstructured code at Meta, but other useful policies are possible. By defining `spawn()` and `spawn_future()` in terms of the more fundamental `nest()`, and leaving the definition of `nest()` to the scope, this paper’s design leaves the set of policies open to extension by user code or future standards.

An async scope's implementation of `nest()`:

— must allow an arbitrary sender to be nested within the scope without eagerly starting the sender;

— must not return a sender that adds new value or error completions to the completions of the sender being nested;

— may fail to nest a new sender by returning an "unnested" sender that completes with `set_stopped` when run without running the sender that failed to nest;

— may fail to nest a new sender by eagerly throwing an exception during the call to `nest()`; and

— is expected to be "cheap" like other sender adaptor objects.

More on these items can be found below in the sections below.

§ 5.1 Definitions

```
namespace { // exposition-only

struct scope-token { // exposition-only
  template <sender Sender>
  sender auto nest(Sender&& s)
      noexcept(is_nothrow_constructible_v<remove_cvref_t<Sender>, Sender>);
};

// TODO: let_with_async_scope will store a copy of the callable and
//       invoke that copy, so perhaps the callability should be assessed
//       on a mutable lvalue reference-qualified type?
template <class Callable>
concept scoped-sender-factory = // exposition-only
    std::invocable<Callable, scope-token> &&
    sender<std::invoke_result_t<Callable, scope-token>>;

template <class Env>
struct spawn-env; // exposition-only

template <class Env>
struct spawn-receiver { // exposition-only
    void set_value() noexcept;
    void set_stopped() noexcept;

    spawn-env<Env> get_env() const noexcept;
};

template <class Env>
struct future-env; // exposition-only

template <valid-completion-signatures Sigs>
struct future-sender; // exposition-only

template <sender Sender, class Env>
using future-sender-t = // exposition-only
    future-sender<completion_signatures_of_t<Sender, future-env<Env>>>;

}

template <class Token, class Sender>
concept async_scope_token =
    copyable<Token> &&
    is_nothrow_move_constructible_v<Token> &&
    is_nothrow_move_assignable_v<Token> &&
    is_nothrow_copy_constructible_v<Token> &&
    is_nothrow_copy_assignable_v<Token> &&
    sender<Sender> &&
    requires(Token token, Sender&& snd) {
      { token.nest(std::forward<Sender>(snd)) } -> sender;
    };

template <class Scope>
concept async_scope =
    requires(Scope scope) {
      { scope.get_token() } -> async_scope_token<decltype(just())>;
      { scope.join() } -> sender;
    };

template <sender Sender, async_scope_token<Sender> Token>
auto nest(Sender&& snd, Token token) noexcept(noexcept(token.nest(std::forward<Sender>(snd))))
    -> decltype(token.nest(std::forward<Sender>(snd)));
```

```
template <sender Sender, async_scope_token<Sender> Token, class Env = empty_env>
  requires sender_to<Sender, spawn-receiver<Env>>
void spawn(Sender&& snd, Token token, Env env = {});

template <sender Sender, async_scope_token<Sender> Token, class Env = empty_env>
future-sender-t<Sender, Env> spawn_future(Sender&& snd, Token token, Env env = {});

template <scoped-sender-factory Callable>
sender auto let_with_async_scope(Callable&& callable)
    noexcept(std::is_nothrow_constructible_v<std::decay_t<Callable>, Callable>);

struct simple_counting_scope {
    simple_counting_scope() noexcept;
    ~simple_counting_scope();

    // simple_counting_scope is immovable and uncopyable
    simple_counting_scope(const simple_counting_scope&) = delete;
    simple_counting_scope(simple_counting_scope&&) = delete;
    simple_counting_scope& operator=(const simple_counting_scope&) = delete;
    simple_counting_scope& operator=(simple_counting_scope&&) = delete;

    template <sender S>
    struct nest-sender; // exposition-only

    struct token {
      template <sender S>
      [[nodiscard]] nest-sender<std::remove_cvref_t<S>> nest(S&& s) const
          noexcept(std::is_nothrow_constructible_v<std::remove_cvref_t<S>, S>);

     private:
      friend simple_counting_scope;

      token(simple_counting_scope* s) noexcept;

      simple_counting_scope* scope; // exposition-only
    };

    token get_token() noexcept;

    void close() noexcept;

    struct join-sender; // exposition-only

    [[nodiscard]] join-sender join() noexcept;
};

struct counting_scope {
    counting_scope() noexcept;
    ~counting_scope();

    // counting_scope is immovable and uncopyable
    counting_scope(const counting_scope&) = delete;
    counting_scope(counting_scope&&) = delete;
    counting_scope& operator=(const counting_scope&) = delete;
    counting_scope& operator=(counting_scope&&) = delete;

    template <sender S>
    struct nest-sender; // exposition-only

    struct token {
      template <sender S>
      [[nodiscard]] nest-sender<std::remove_cvref_t<S>> nest(S&& s) const
          noexcept(std::is_nothrow_constructible_v<std::remove_cvref_t<S>, S>);

     private:
      friend counting_scope;

      token(counting_scope* s) noexcept;

      counting_scope* scope; // exposition-only
    };

    token get_token() noexcept;

    void close() noexcept;

    void request_stop() noexcept;

    struct join-sender; // exposition-only
```

```
    [[nodiscard]] join-sender join() noexcept;
};
```

§ 5.2 execution::async_scope_token

```
template <class Token, class Sender>
concept async_scope_token =
    copyable<Token> &&
    is_nothrow_move_constructible_v<Token> &&
    is_nothrow_move_assignable_v<Token> &&
    is_nothrow_copy_constructible_v<Token> &&
    is_nothrow_copy_assignable_v<Token> &&
    sender<Sender> &&
    requires(Token token, Sender&& snd) {
      { token.nest(std::forward<Sender>(snd)) } -> sender;
    };
```

An async scope token is a non-owning handle to an async scope. The `nest()` method on a token attempts to associate its input sender with the handle's async scope in a scope-defined way. See `execution::nest` for the semantics of `nest()`.

An async scope token behaves like a pointer-to-async-scope; tokens are no-throw copyable and movable, and it is undefined behaviour to invoke `nest()` on a token that has outlived its scope.

§ 5.3 execution::async_scope

```
template <class Scope>
concept async_scope =
    requires(Scope scope) {
      { scope.get_token() } -> async_scope_token<decltype(just())>;
      { scope.join() } -> sender;
    };
```

As described above, an async scope is a type that implements a bookkeeping policy for senders. If a given type, `Scope`, satisifies `async_scope<Scope>` then, for a value, `scope`, of type `Scope`:

— `scope.get_token()` returns an async scope token, `t`, whose `nest()` method tries to associate the input sender with `scope`; and

— `scope.join()` returns a sender, `s`, such that,

  — `s` may only complete with `set_value()`;

  — connecting and starting `s` requests that the scope arrange not to have any outstanding associated senders "soon" (the interpretation of this request and the definition of "soon" are left up to the definition of `Scope`); and

  — once `s` has completed, no new senders may become associated with `scope` (the means by which this guarantee is provided is left up to the definition of `Scope`).

§ 5.4 execution::nest

```
template <sender Sender, async_scope_token<Sender> Token>
auto nest(Sender&& snd, Token token) noexcept(noexcept(token.nest(std::forward<Sender>(snd))))
    -> decltype(token.nest(std::forward<Sender>(snd)));
```

Attempts to associate the given sender with the given scope token's scope in a scope-defined way. When successful, the return value is an "associated sender" with the same behaviour and possible completions as the input sender, plus the additional, scope-specific behaviours that are necessary to implement the scope's bookkeeping policy. When the attempt fails, `nest()` must either eagerly throw an exception, or return a "unassociated sender" that, when started, unconditionally completes with `set_stopped()`.

A call to `nest()` does not start the given sender and is not expected to incur allocations.

When `nest()` returns an associated sender:

— connecting and starting the associated sender connects and starts the given sender; and

— the associated sender has exactly the same completions as the input sender.

When `nest()` returns an unassociated sender:

— the input sender is discarded and will never be connected or started; and

— the unassociated sender must only complete with `set_stopped()`.

Regardless of whether the returned sender is associated or unassociated, it is multi-shot if the input sender is multi-shot and single-shot otherwise.

§ 5.5 execution::spawn()

```
namespace { // exposition-only

template <class Env>
struct spawn-env; // exposition-only

template <class Env>
struct spawn-receiver { // exposition-only
    void set_value() noexcept;
    void set_stopped() noexcept;

    spawn-env<Env> get_env() const noexcept;
};

}

template <sender Sender, async_scope_token<Sender> Token, class Env = empty_env>
  requires sender_to<Sender, spawn-receiver<Env>>
void spawn(Sender&& snd, Token token, Env env = {});
```

Invokes `nest(std::forward<Sender>(snd), token)` to associate the given sender with the given token's scope and then eagerly starts the resulting sender.

Starting the nested sender involves a dynamic allocation of the sender's *operation-state*. The following algorithm determines which *Allocator* to use for this allocation:

— If `get_allocator(env)` is valid and returns an *Allocator* then choose that *Allocator*.

— Otherwise, if `get_allocator(get_env(snd))` is valid and returns an *Allocator* then choose that *Allocator*.

— Otherwise, choose `std::allocator<>`.

The *operation-state* is constructed by connecting the nested sender to a *spawn-receiver*. The *operation-state* is destroyed and deallocated after the spawned sender completes.

A *spawn-receiver*, `sr`, responds to `get_env(sr)` with an instance of a *spawn-env*`<Env>`, `senv`. The result of `get_allocator(senv)` is a copy of the *Allocator* used to allocate the *operation-state*. For all other queries, `Q`, the result of `Q(senv)` is `Q(env)`.

This is similar to `start_detached()` from [P2300R7], but the scope may observe and participate in the lifecycle of the work described by the sender. The `counting_scope` described in this paper uses this opportunity to keep a count of nested senders that haven't finished, and to prevent new work from being started once the `counting_scope`'s *join-sender* has been started.

The given sender must complete with `set_value()` or `set_stopped()` and may not complete with an error; the user must explicitly handle the errors that might appear as part of the *sender-expression* passed to `spawn()`.

User expectations will be that `spawn()` is asynchronous and so, to uphold the principle of least surprise, `spawn()` should only be given non-blocking senders. Using `spawn()` with a sender generated by `on(sched, `*blocking-sender*`)` is a very useful pattern in this context.

NOTE: A query for non-blocking start will allow `spawn()` to be constrained to require non-blocking start.

Usage example:

```
...
for (int i = 0; i < 100; i++)
    spawn(on(sched, some_work(i)), scope.get_token());
```

§ 5.6 execution::spawn_future()

```
namespace { // exposition-only

template <class Env>
struct future-env; // exposition-only

template <valid-completion-signatures Sigs>
struct future-sender; // exposition-only

template <sender Sender, class Env>
using future-sender-t = // exposition-only
    future-sender<completion_signatures_of_t<Sender, future-env<Env>>>;

}

template <sender Sender, async_scope_token<Sender> Token, class Env = empty_env>
future-sender-t<Sender, Env> spawn_future(Sender&& snd, Token token, Env env = {});
```

Invokes `nest(std::forward<Sender>(snd), token)` to associate the given sender with the given token's scope, eagerly starts the resulting sender, and returns a *`future-sender`* that provides access to the result of the given sender.

Similar to `spawn()`, starting the nested sender involves a dynamic allocation of some state. `spawn_future()` chooses an *Allocator* for this allocation in the same way `spawn()` does: use the result of `get_allocator(env)` if that is a valid expression, otherwise use the result of `get_allocator(get_env(snd))` if that is a valid expression, otherwise use a `std::allocator<>`.

Unlike `spawn()`, the dynamically allocated state contains more than just an *`operation-state`* for the nested sender; the state must also contain storage for the result of the nested sender, however it eventually completes, and synchronization facilities for resolving the race between the nested sender's production of its result and the returned sender's consumption or abandonment of that result.

Also unlike `spawn()`, `spawn_future()` returns a *`future-sender`* rather than `void`. The returned sender, `fs`, is a handle to the spawned work that can be used to consume or abandon the result of that work. When `fs` is connected and started, it waits for the spawned sender to complete and then completes itself with the spawned sender's result. If `fs` is destroyed before being connected, or if `fs` *is* connected but then the resulting *`operation-state`* is destroyed before being started, then a stop request is sent to the spawned sender in an effort to short-circuit the computation of a result that will not be observed. If `fs` receives a stop request from its receiver before the spawned sender completes, the stop request is forwarded to the spawned sender and then `fs` completes; if the spawned sender happens to complete between `fs` forwarding the stop request and completing itself then `fs` may complete with the result of the spawned sender as if the stop request was never received but, otherwise, `fs` completes with `stopped` and the result of the spawned sender is ignored. The completion signatures of `fs` include `set_stopped()` and all the completion signatures of the spawned sender.

The receiver, `fr`, that is connected to the nested sender responds to `get_env(fr)` with an instance of *`future-env`*`<Env>`, `fenv`. The result of `get_allocator(fenv)` is a copy of the *Allocator* used to allocate the dynamically allocated state. The result of `get_stop_token(fenv)` is a stop token that will be "triggered" (i.e. signal that stop is requested) by the returned *`future-sender`* when it is dropped or receives a stop request itself. For all other queries, `Q`, the result of `Q(fenv)` is `Q(env)`.

This is similar to `ensure_started()` from [P2300R7], but the scope may observe and participate in the lifecycle of the work described by the sender. The `counting_scope` described in this paper uses this opportunity to keep a count of nested senders that haven't finished, and to prevent new work from being started once the `counting_scope`'s *`join-sender`* has been started.

Unlike `spawn()`, the sender given to `spawn_future()` is not constrained on a given shape. It may send different types of values, and it can complete with errors.

NOTE: there is a race between the completion of the given sender and the start of the returned sender. The spawned sender and the returned *`future-sender`* use the synchronization facilities in the dynamically allocated state to resolve this race.

Cancelling the returned sender requests cancellation of the given sender, `snd`, but does not affect any other senders.

Usage example:

```
...
sender auto snd = spawn_future(on(sched, key_work()), scope) | then(continue_fun);
for (int i = 0; i < 10; i++)
    spawn(on(sched, other_work(i)), scope);
return when_all(scope.join(), std::move(snd));
```

§ 5.7 `execution::simple_counting_scope`

```
struct simple_counting_scope {
    simple_counting_scope() noexcept;
    ~simple_counting_scope();

    // simple_counting_scope is immovable and uncopyable
    simple_counting_scope(const simple_counting_scope&) = delete;
    simple_counting_scope(simple_counting_scope&&) = delete;
    simple_counting_scope& operator=(const simple_counting_scope&) = delete;
    simple_counting_scope& operator=(simple_counting_scope&&) = delete;

    template <sender S>
    struct nest-sender; // exposition-only

    struct token {
      template <sender S>
      [[nodiscard]] nest-sender<std::remove_cvref_t<S>> nest(S&& s) const
          noexcept(std::is_nothrow_constructible_v<std::remove_cvref_t<S>, S>);

     private:
      friend simple_counting_scope;

      token(simple_counting_scope* s) noexcept;

      simple_counting_scope* scope; // exposition-only
    };
```

```
    token get_token() noexcept;

    void close() noexcept;

    struct join-sender; // exposition-only

    [[nodiscard]] join-sender join() noexcept;
};
```

A `simple_counting_scope` maintains a count of outstanding operations and goes through several states durings its lifetime:

— unused

— open

— closed

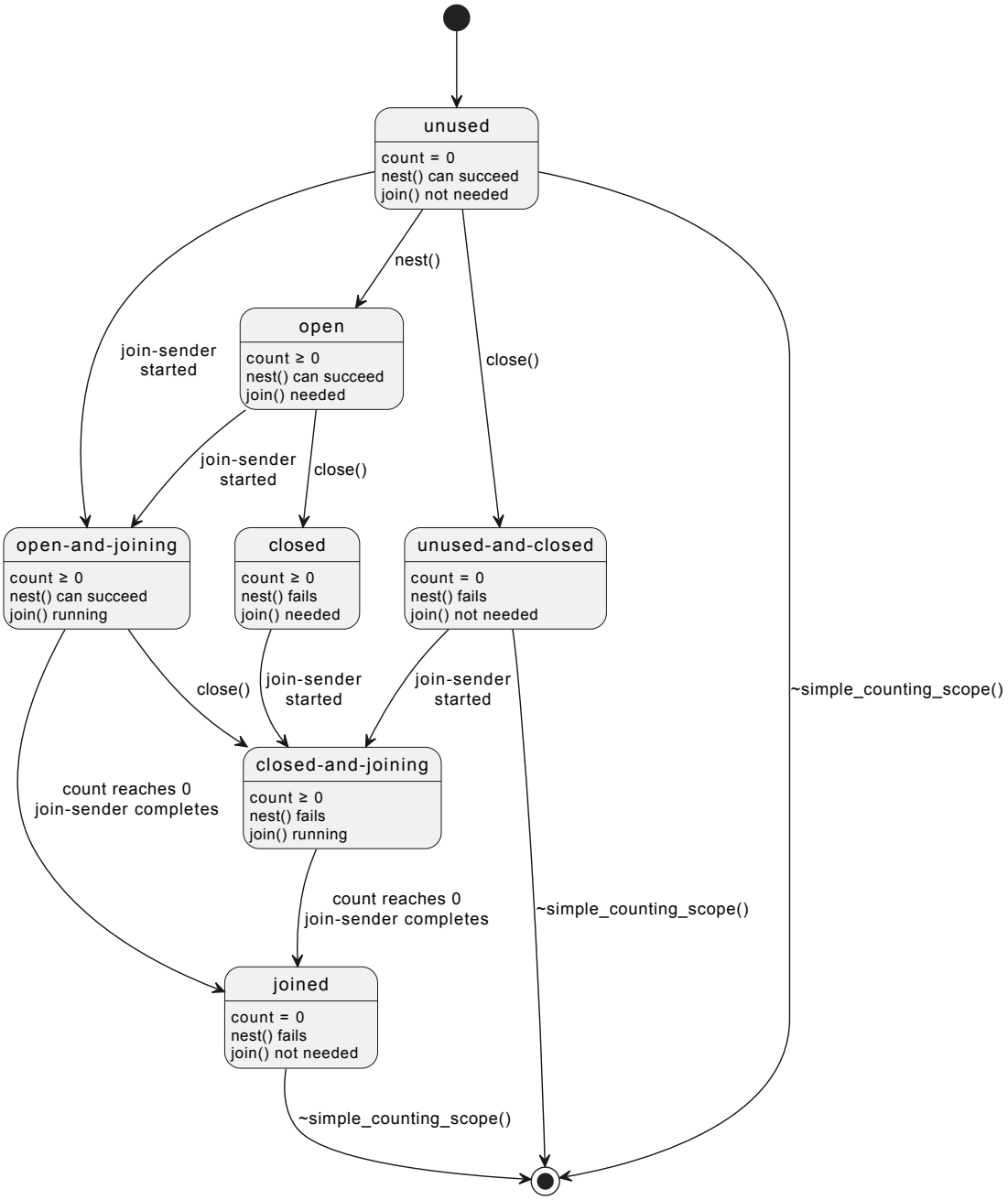
— open-and-joining

— closed-and-joining

— unused-and-closed

— joined

The following diagram illustrates the `simple_counting_scope`'s state machine:



Note: a scope is "open" if its current state is unused, open, or open-and-joining; a scope is "closed" if its current state is closed, unused-and-closed, closed-and-joining, or joined.

Instances start in the unused state after being constructed. This is the only time the scope's state can be set to unused. When the `simple_counting_scope` destructor starts, the scope must be in the unused, unused-and-closed, or joined state; otherwise, the destructor invokes `std::terminate()`. Permitting destruction when the scope is in the unused or unused-and-closed state ensures that instances of `simple_counting_scope` can be used safely as data-members while preserving structured functionality.

Connecting and starting a *`join-sender`* returned from `join()` moves the scope to either the open-and-joining or closed-and-joining state. Merely calling `join()` or connecting the *`join-sender`* does not change the scope's state—the *`operation-state`* must be started to effect the state change. A started *`join-sender`* completes when the scope's count of outstanding operations reaches zero, at which point the scope transitions to the joined state.

Calling `close()` on a `simple_counting_scope` moves the scope to the closed, unused-and-closed, or closed-and-joining state, and causes all future calls to `nest()` to fail.

Any call to `nest()` may throw an exception if copying or moving the input sender into the returned *`nest-sender`* throws an exception. `nest()` provides the Strong Exception Guarantee so the scope's state is left unchanged if an exception is thrown while constructing the returned *`nest-sender`*.

Assuming `nest()` does not throw:

— While a scope is in the unused, open, or open-and-joining state, calls to `nest()` succeed by returning an "associated sender" (see below) and incrementing the scope's count of outstanding operations *before returning*.

— While a scope is in the closed, unused-and-closed, closed-and-joining, or joined state, calls to `nest()` fail by returning an "unassociated sender" (see below). Failed calls to `nest()` do *not* increment the scope's count of outstanding operations.

While a scope is open, calls to `nest()` that return normally will have incremented the scope's count of oustanding operations. In this case, the resulting *`nest-sender`* is an associated sender that acts like an RAII handle: the scope's internal count is incremented when the sender is created and decremented when the sender is "done with the scope", which happens when the sender or its *`operation-state`* is destroyed. Moving a *`nest-sender`* transfers responsibility for decrementing the count from the old instance to the new one. Copying an associated *`nest-sender`* is permitted if the sender it's wrapping is copyable, but the copy may "fail" since copying requires incrementing the scope's count, which is only allowed when the scope is open; if copying fails, the new sender is an unassociated sender that behaves as if it were the result of a failed call to `nest()`.

While a scope is closed, calls to `nest()` that return normally will have failed to increment the scope's count of outstanding operations or otherwise change the scope's state. In this case, the resulting *`nest-sender`* is an unassociated sender. Unassociated *`nest-senders`* do not have a reference to the scope they came from and always complete with `stopped` when connected and started. Copying or moving an unassociated sender produces another unassociated sender.

Under the standard assumption that the arguments to `nest()` are and remain valid while evaluating `nest()`, it is always safe to invoke any supported operation on the returned *`nest-sender`*.

The state transitions of a `simple_counting_scope` mean that it can be used to protect asynchronous work from use-after-free errors. Given a resource, `res`, and a `simple_counting_scope`, `scope`, obeying the following policy is enough to ensure that there are no attempts to use `res` after its lifetime ends:

— all senders that refer to `res` are nested within `scope`; and

— `scope` is destroyed (and therefore in the joined, unused, or unused-and-closed state) before `res` is destroyed.

It is safe to destroy a scope in the unused or unusued-and-closed state because there can't be any work referring to the resources protected by the scope.

A `simple_counting_scope` is uncopyable and immovable so its copy and move operators are explicitly deleted. `simple_counting_scope` could be made movable but it would cost an allocation so this is not proposed.

§ 5.7.1 `simple_counting_scope::simple_counting_scope()`

```
simple_counting_scope::simple_counting_scope() noexcept;
```

Initializes a `simple_counting_scope` in the unused state with the count of outstanding operations set to zero.

§ 5.7.2 `simple_counting_scope::~simple_counting_scope()`

```
simple_counting_scope::~simple_counting_scope();
```

Checks that the `simple_counting_scope` is in the joined, unused, or unused-and-closed state and invokes `std::terminate()` if not.

§ 5.7.3 `simple_counting_scope::get_token()`

```
simple_counting_scope::token get_token() noexcept;
```

Returns a `simple_counting_scope::token` referring to the current scope, as if by invoking `token{this}`.

§ 5.7.4 `simple_counting_scope::close()`

```
void close() noexcept;
```

Moves the scope to the closed, unused-and-closed, or closed-and-joining state. After a call to `close()`, all future calls to `nest()` that return normally return unassociated senders.

§ 5.7.5 `simple_counting_scope::join()`

```
struct join-sender; // exposition-only

[[nodiscard]] join-sender join() noexcept;
```

Returns a *`join-sender`*. When the *`join-sender`* is connected to a receiver, `r`, it produces an *`operation-state`*, `o`. When `o` is started, the scope moves to either the open-and-joining or closed-and-joining state. `o` completes with `set_value()` when the scope moves to the joined state, which happens when the scope's count of outstanding senders drops to zero. `o` may complete synchronously if it happens to observe that the count of outstanding senders is already zero when started; otherwise, `o` completes on the execution context associated with the scheduler in its receiver's environment by asking its receiver, `r`, for a scheduler, `sch`, with `get_scheduler(get_env(r))` and then starting

the sender returned from `schedule(sch)`. This requirement to complete on the receiver's scheduler restricts which receivers a *`join-sender`* may be connected to in exchange for determinism; the alternative would have the *`join-sender`* completing on the execution context of whichever nested operation happens to be the last one to complete.

5.7.6 `simple_counting_scope::token::nest()`

```
template <sender S>
struct nest-sender; // exposition-only

template <sender S>
[[nodiscard]] nest-sender<std::remove_cvref_t<S>> nest(S&& s) const noexcept(
        std::is_nothrow_constructible_v<std::remove_cvref_t<S>, S>);
```

Attempts to return an associated *`nest-sender`* constructed from `s`. The attempt will be successful if and only if:

— copying or moving (as appropriate) `s` into the *`nest-sender`* does not throw an exception, and

— the token's scope's state can be successfully updated as described below.

If construction of the *`nest-sender`* throws, the scope's state is left unchanged. Otherwise, the following atomic state change is attempted on the token's scope:

— increment the scope's count of outstanding operations; and

— move the scope to the open state if it was in the unused state.

The atomic state change succeeds if the scope is observed to be in the unused, open, or open-and-joining state; otherwise it fails.

If the atomic state change fails then the return value is an unassociated *`nest-sender`*.

An associated *`nest-sender`* is a kind of RAII handle to the scope; it is responsible for decrementing the scope's count of outstanding senders in its destructor unless that responsibility is first given to some other object. Move-construction and move-assignment transfer the decrement responsibility to the destination instance. Connecting an instance to a receiver transfers the decrement responsibility to the resulting *`operation-state`*, which must meet the responsibility when it destroys its "child operation" (i.e. the *`operation-state`* constructed when connecting the sender, `s`, that was originally passed to `nest()`); it's expected that the child operation will be destroyed as a side effect of the *`nest-sender`*'s *`operation-state`*'s destructor.

Note: the timing of when an *`operation-state`* decrements the scope's count is chosen to avoid exposing user code to dangling references. Decrementing the scope's count may move the scope to the joined state, which would allow the waiting *`join-sender`* to complete, potentially leading to the destruction of a resource protected by the scope. In general, it's possible that the *`nest-sender`*'s receiver or the child operation's destructor may dereference pointers to the protected resource so their execution must be completed before the scope moves to the joined state.

Whenever the balancing decrement happens, it's possible that the scope has transitioned to the open-and-joining or closed-and-joining state since the *`nest-sender`* was constructed, which means that there is a *`join-sender`* waiting to complete. If the decrement brings the count of outstanding operations to zero then the waiting *`join-sender`* must be notified that the scope is now joined and the sender can complete.

A call to `nest()` does not start the given sender. A call to `nest()` is not expected to incur allocations other than whatever might be required to move or copy `s`.

Similar to `spawn_future()`, `nest()` doesn't constrain the input sender to any specific shape. Any type of sender is accepted.

As `nest()` does not immediately start the given work, it is ok to pass in blocking senders.

Usage example:

```
sender auto example(simple_counting_scope::token token, scheduler auto sched) {
  sender auto snd = nest(key_work(), token);

  for (int i = 0; i < 10; i++)
    spawn(on(sched, other_work(i)), token);

  return on(sched, std::move(snd));
}
```

5.8 `execution::counting_scope`

```
struct counting_scope {
    counting_scope() noexcept;
    ~counting_scope();

    // counting_scope is immovable and uncopyable
    counting_scope(const counting_scope&) = delete;
    counting_scope(counting_scope&&) = delete;
```

```
    counting_scope& operator=(const counting_scope&) = delete;
    counting_scope& operator=(counting_scope&&) = delete;

    template <sender S>
    struct nest-sender; // exposition-only

    struct token {
      template <sender S>
      [[nodiscard]] nest-sender<std::remove_cvref_t<S>> nest(S&& s) const
          noexcept(std::is_nothrow_constructible_v<std::remove_cvref_t<S>, S>);

     private:
      friend counting_scope;

      token(counting_scope* s) noexcept;

      counting_scope* scope; // exposition-only
    };

    token get_token() noexcept;

    void close() noexcept;

    void request_stop() noexcept;

    struct join-sender; // exposition-only

    [[nodiscard]] join-sender join() noexcept;
};
```

A `counting_scope` augments a `simple_counting_scope` with a stop source and gives to each of its associated *`nest-senders`* a stop token from its stop source. This extension of `simple_counting_scope` allows a `counting_scope` to request stop on all of its outstanding operations by requesting stop on its stop source.

Assuming an exposition-only *`stop_when(sender auto&&, stoppable_token auto)`* (explained below), `counting_scope` behaves as if it were implemented like so:

```
struct counting_scope {
  struct token {
    template <sender S>
    sender auto nest(S&& snd) const
        noexcept(std::is_nothrow_constructible_v<std::remove_cvref_t<S>, S>) {
      return std::forward<Sender>(snd)
          | stop_when(scope_->source_.get_token())
          | ex::nest(scope_->scope_.get_token());
    }

   private:
    friend counting_scope;

    explicit token(counting_scope* scope) noexcept
      : scope_(scope) {}

    counting_scope* scope_;
  };

  token get_token() noexcept {
    return token{this};
  }

  void close() noexcept {
    return scope_.close();
  }

  void request_stop() noexcept {
    source_.request_stop();
  }

  sender auto join() noexcept {
    return scope_.join();
  }

 private:
  simple_counting_scope scope_;
  inplace_stop_source source_;
};
```

stop_when(sender auto&& snd, stoppable_token auto stoken) is an exposition-only sender algorithm that maps its input sender, `snd`, to an output sender, `osnd`, such that, when `osnd` is connected to a receiver, `r`, the resulting *operation-state* behaves the same as connecting the original sender, `snd`, to `r`, except that `snd` will receive a stop request when either the token returned from `get_stop_token(r)` receives a stop request or when `stoken` receives a stop request.

Other than the use of *stop_when()* in `counting_scope::token::nest()` and the addition of `request_stop()` to the interface, `counting_scope` has the same behavior and lifecycle as `simple_counting_scope`.

5.8.1 `counting_scope::counting_scope()`

```
counting_scope::counting_scope() noexcept;
```

Initializes a `counting_scope` in the unused state with the count of outstanding operations set to zero.

5.8.2 `counting_scope::~counting_scope()`

```
counting_scope::~counting_scope();
```

Checks that the `counting_scope` is in the joined, unused, or unused-and-closed state and invokes `std::terminate()` if not.

5.8.3 `counting_scope::get_token()`

```
counting_scope::token get_token() noexcept;
```

Returns a `counting_scope::token` referring to the current scope, as if by invoking `token{this}`.

5.8.4 `counting_scope::close()`

```
void close() noexcept;
```

Moves the scope to the closed, unused-and-closed, or closed-and-joining state. After a call to `close()`, all future calls to `nest()` that return normally return unassociated senders.

5.8.5 `counting_scope::request_stop()`

```
void request_stop() noexcept;
```

Requests stop on the scope's internal stop source. Since all senders nested within the scope have been given stop tokens from this internal stop source, the effect is to send stop requests to all outstanding (and future) nested operations.

5.8.6 `counting_scope::join()`

```
struct join-sender; // exposition-only

[[nodiscard]] join-sender join() noexcept;
```

Returns a *join-sender* that behaves the same as the result of `simple_counting_scope::join()`. Connecting and starting the *join-sender* moves the scope to the open-and-joining or closed-and-joining state; the *join-sender* completes when the scope's count of outstanding operations drops to zero, at which point the scope moves to the joined state.

5.8.7 `counting_scope::token::nest()`

```
template <sender S>
struct nest-sender; // exposition-only

template <sender S>
[[nodiscard]] nest-sender<std::remove_cvref_t<S>> nest(S&& s) const noexcept(
        std::is_nothrow_constructible_v<std::remove_cvref_t<S>, S>);
```

Attempts to return an associated *nest-sender* constructed from `s` following the same algorithm as `simple_counting_scope::token::nest()`, with the addition that senders associated with a `counting_scope` receive stop requests *both* from their (eventual) receivers *and* from the `counting_scope`'s internal stop source.

5.9 When to use `counting_scope` vs [P3296R0]'s `let_with_async_scope`

Although `counting_scope` and `let_with_async_scope` have overlapping use-cases, we specifically designed the two facilities to address separate problems. In short, `counting_scope` is best used in an unstructured context and `let_with_async_scope` is best used in a structured context.

We define "unstructured context" as:

— a place where using `sync_wait` would be inappropriate,

— and you can't "solve by induction" (i.e you're not in an async context where you can start the sender by "awaiting" it)

`counting_scope` should be used when you have a sender you want to start in an unstructured context. In this case, `spawn(sender, scope.get_token())` would be the preferred way of starting asynchronous work. `scope.join()` needs to be called before the owning object's destruction in order to ensure that the object's lifetime lives at least until all asynchronous work completes. Note that exception safety needs to be handled explicitly in the use of `counting_scope`.

`let_with_async_scope` returns a sender, and therefore can only be started in one of 3 ways:

1. `sync_wait`
2. `spawn` on a `counting_scope`
3. `co_await`

`let_with_async_scope` will manage the scope for you, ensuring that the managed scope is always joined before `let_with_async_scope` completes. The algorithm frees the user from having to manage the coupling between the lifetimes of the managed scope and the resource(s) it protects with the limitation that the nested work must be fully structured. This behavior is a feature, since the scope being managed by `let_with_async_scope` is intended to live only until the sender completes. This also means that `let_with_async_scope` will be exception safe by default.

§ 6 Design considerations

§ 6.1 Shape of the given sender

§ 6.1.1 Constraints on `set_value()`

It makes sense for `spawn_future()` and `nest()` to accept senders with any type of completion signatures. The caller gets back a sender that can be chained with other senders, and it doesn't make sense to restrict the shape of this sender.

The same reasoning doesn't necessarily follow for `spawn()` as it returns `void` and the result of the spawned sender is dropped. There are two main alternatives:

— do not constrain the shape of the input sender (i.e., dropping the results of the computation)

— constrain the shape of the input sender

The current proposal goes with the second alternative. The main reason is to make it more difficult and explicit to silently drop results. The caller can always transform the input sender before passing it to `spawn()` to drop the values manually.

> **Chosen:** `spawn()` accepts only senders that advertise `set_value()` (without any parameters) in the completion signatures.

§ 6.1.2 Handling errors in `spawn()`

The current proposal does not accept senders that can complete with error given to `spawn()`. This will prevent accidental error scenarios that will terminate the application. The user must deal with all possible errors before passing the sender to `spawn()`. i.e., error handling must be explicit.

Another alternative considered was to call `std::terminate()` when the sender completes with error.

Another alternative is to silently drop the errors when receiving them. This is considered bad practice, as it will often lead to first spotting bugs in production.

> **Chosen:** `spawn()` accepts only senders that do not call `set_error()`. Explicit error handling is preferred over stopping the application, and over silently ignoring the error.

§ 6.1.3 Handling stop signals in `spawn()`

Similar to the error case, we have the alternative of allowing or forbidding `set_stopped()` as a completion signal. Because the goal of `counting_scope` is to track the lifetime of the work started through it, it shouldn't matter whether that the work completed with success or by being stopped. As it is assumed that sending the stop signal is the result of an explicit choice, it makes sense to allow senders that can terminate with `set_stopped()`.

The alternative would require transforming the sender before passing it to spawn, something like `spawn(std::move(snd) | let_stopped(just), s.get_token())`. This is considered boilerplate and not helpful, as the stopped scenarios should be implicit, and not require handling.

> **Chosen:** `spawn()` accepts senders that complete with `set_stopped()`.

§ 6.1.4 No shape restrictions for the senders passed to `spawn_future()` and `nest()`

Similarly to `spawn()`, we can constrain `spawn_future()` and `nest()` to accept only a limited set of senders. But, because we can attach continuations for these senders, we would be limiting the functionality that can be expressed. For example, the continuation can handle different types of values and errors.

> **Chosen:** `spawn_future()` and `nest()` accept senders with any completion signatures.

6.2 P2300's `start_detached()`

The `spawn()` algorithm in this paper can be used as a replacement for `start_detached` proposed in [P2300R7]. Essentially it does the same thing, but it also provides the given scope the opportunity to apply its bookkeeping policy to the given sender, which, in the case of `counting_scope`, ensures the program can wait for spawned work to complete before destroying any resources references by that work.

6.3 P2300's `ensure_started()`

The `spawn_future()` algorithm in this paper can be used as a replacement for `ensure_started` proposed in [P2300R7]. Essentially it does the same thing, but it also provides the given scope the opportunity to apply its bookkeeping policy to the given sender, which, in the case of `counting_scope`, ensures the program can wait for spawned work to complete before destroying any resources references by that work.

6.4 Supporting the pipe operator

This paper doesn't support the pipe operator to be used in conjunction with `spawn()` and `spawn_future()`. One might think that it is useful to write code like the following:

```
std::move(snd1) | spawn(s); // returns void
sender auto snd3 = std::move(snd2) | spawn_future(s) | then(...);
```

In [P2300R7] sender consumers do not have support for the pipe operator. As `spawn()` works similarly to `start_detached()` from [P2300R7], which is a sender consumer, if we follow the same rationale, it makes sense not to support the pipe operator for `spawn()`.

On the other hand, `spawn_future()` is not a sender consumer, thus we might have considered adding pipe operator to it.

On the third hand, Unifex supports the pipe operator for both of its equivalent algorithms (`unifex::spawn_detached()` and `unifex::spawn_future()`) and Unifex users have not been confused by this choice.

To keep consistency with `spawn()` this paper doesn't support pipe operator for `spawn_future()`.

7 Naming

As is often true, naming is a difficult task.

7.1 `nest()`

This provides a way to build a sender that is associated with a "scope", which is a type that implements and enforces some bookkeeping policy regarding the senders nested within it. `nest()` does not allocate state, call connect, or call start. `nest()` is the basis operation for async scopes. `spawn()` and `spawn_future()` use `nest()` to associate a given sender with a given scope, and then they allocate, connect, and start the resulting sender.

It would be good for the name to indicate that it is a simple operation (insert, add, embed, extend might communicate allocation, which `nest()` does not do).

alternatives: `wrap()`, `attach()`

7.2 `async_scope`

This is a concept that is satisfied by types that support nesting senders within themselves. It is primarily useful for constraining the arguments to `spawn()` and `spawn_future()` to give useful error messages for invalid invocations.

Since concepts don't support existential quantifiers and thus can't express "type `T` is an `async_scope` if there exists a sender, `s`, for which `t.nest(s)` is valid", the `async_scope` concept must be parameterized on both the type of the scope and the type of some particular sender and thus describes whether *this* scope type is an `async_scope` in combination with *this* sender type. Given this limitation, perhaps the name should convey something about the fact that it is checking the relationship between two types rather than checking something about the scope's type alone. Nothing satisfying comes to mind.

alternatives: don't name it and leave it as *`exposition-only`*

7.3 `spawn()`

This provides a way to start a sender that produces `void` and to associate the resulting async work with an async scope that can implement a bookkeeping policy that may help ensure the async work is complete before destroying any resources it is using. This allocates, connects, and starts the given sender.

It would be good for the name to indicate that it is an expensive operation.

alternatives: `connect_and_start()`, `spawn_detached()`, `fire_and_remember()`

7.4 `spawn_future()`

This provides a way to start work and later ask for the result. This will allocate, connect, and start the given sender, while resolving the race (using synchronization primitives) between the completion of the given sender and the start of the returned sender. Since the type of the receiver supplied to the result sender is not known when the given sender starts, the receiver will be type-erased when it is connected.

It would be good for the name to be ugly, to indicate that it is a more expensive operation than `spawn()`.

alternatives: `spawn_with_result()`

7.5 `counting_scope`

A `counting_scope` represents the root of a set of nested lifetimes.

One mental model for this is a semaphore. It tracks a count of lifetimes and fires an event when the count reaches 0.

Another mental model for this is block syntax. `{}` represents the root of a set of lifetimes of locals and temporaries and nested blocks.

Another mental model for this is a container. This is the least accurate model. This container is a value that does not contain values. This container contains a set of active senders (an active sender is not a value, it is an operation).

alternatives: `async_scope`

7.5.1 `counting_scope::join()`

This method returns a sender that, when started, prevents new senders from being nested within the scope and then waits for the scope's count of outstanding senders to drop to zero before completing. It is somewhat analogous to `std::thread::join()` but does not block.

`join()` must be invoked, and the returned sender must be connected, started, and completed, before the scope may be destroyed so it may be useful to convey some of this importance in the name, although `std::thread` has similar requirements for its `join()`.

`join()` is the biggest wart in this design; the need to manually manage the end of a scope's lifetime stands out as less-than-ideal in C++, and there is some real risk that users will write deadlocks with `join()` so perhaps `join()` should have a name that conveys danger.

alternatives: `complete()`, `close()`

7.6 "Token" as in `async_scope_token` and `counting_scope::token`

The first several revisions of this paper did not separate the responsibilities of an async scope into the scope and its tokens. Revision 3 introduces this split to help separate the lifetime-management interface from the nesting interface, which the authors expect to help avoid deadlocks (e.g. it's harder with the new design to nest the *`join-sender`* within the scope being joined). The name "token" was chosen by analogy to "stop source" and "stop token", which provides a similar split of responsibilities.

alternatives: "handle", "ref" (as a contraction of "reference")

8 Acknowledgements

Thanks to Andrew Royes for unwavering support for the development and deployment of Unifex at Meta and for recognizing the importance of contributing this paper to the C++ Standard.

Thanks to Eric Niebler for the encouragement and support it took to get this paper published.

9 References

[Dahl72] O.-J. Dahl, E. W. Dijkstra, and C. A. R. Hoare. *Structured Programming*. Academic Press Ltd., 1972.

[`folly::coro`] folly::coro.
https://github.com/facebook/folly/tree/main/folly/experimental/coro

[`folly::coro::AsyncScope`] folly::coro::AsyncScope.
https://github.com/facebook/folly/blob/main/folly/experimental/coro/AsyncScope.h

[io_uring HTTP server] io_uring HTTP server.
https://github.com/facebookexperimental/libunifex/blob/main/examples/linux/http_server_io_uring_test.cpp
[libunifex] libunifex.
https://github.com/facebookexperimental/libunifex/
[P2079R2] Lee Howes, Ruslan Arutyunyan, Michael Voss. 2022-01-15. System execution context.
https://wg21.link/p2079r2
[P2300R7] Eric Niebler, Michał Dominiak, Georgy Evtushenko, Lewis Baker, Lucian Radu Teodorescu, Lee Howes, Kirk Shoop, Michael Garland, Bryce Adelstein Lelbach. 2023-04-21. `std::execution`.
https://wg21.link/p2300r7
[P3296R0] Anthony Williams. let_async_scope.
https://wg21.link/p3296r0
[rsys] A smaller, faster video calling library for our apps.
https://engineering.fb.com/2020/12/21/video-engineering/rsys/
[`unifex::v1::async_scope`] unifex::v1::async_scope.
https://github.com/facebookexperimental/libunifex/blob/main/include/unifex/v1/async_scope.hpp
[`unifex::v2::async_scope`] unifex::v2::async_scope.
https://github.com/facebookexperimental/libunifex/blob/main/include/unifex/v2/async_scope.hpp